

ConOps Retrieval Pipeline

Etapa 1 — Coleta Automática de Dados
Web Crawling e recuperação de documentos de concepção

Projeto: Busca Automática e Classificação Semântica de
Documentos de Concepção de Sistemas Críticos para Utilização no Processo STPA

Autor: Arthur Miele Malveste

Orientadora: Profa. Dra. Ana Estela Antunes da Silva

Coorientador: Prof. Dr. Pedro Ivo Garcia Nunes

Documentação técnica do sistema

Agosto de 2026

Sumário

1. Para que serve este sistema

- O problema que ele resolve
- Onde ele se encaixa no pipeline da IC
- O que é um ConOps

2. Glossário — toda a nomenclatura explicada

3. Visão geral da arquitetura

- As quatro camadas
- As três vias de acesso a uma fonte

4. Passo a passo: o que acontece quando você roda um comando

5. Arquitetura detalhada, módulo por módulo

- Núcleo (`crawler/core/`)
- Motor de rastreamento (`crawler/engine/`)
- Adaptadores (`crawler/adapters/`)
- Interface de linha de comando

6. Conceitos que costumam confundir

- Por que existem duas filas
- Três identidades diferentes
- Descobrir ≠ baixar

7. O pré-filtro léxico e as quatro faixas

8. O rastreamento focado (*Web Crawling semântico*)

9. Inventário de fontes e situação de cada uma

10. Conformidade legal e ética

11. O que já está pronto — pontos positivos

12. Limitações e pontos negativos

13. Próximos passos priorizados

14. Referência rápida de comandos

1. Para que serve este sistema

1.1 O problema que ele resolve

A **STPA** (*System-Theoretic Process Analysis*) é uma técnica de análise de perigos que trata segurança como um problema de controle inadequado entre componentes de um sistema. Sua etapa inicial exige identificar perdas, perigos e restrições de segurança a partir de bases extensas de documentos técnicos não estruturados.

Hoje essa busca e triagem é **predominantemente manual**: um engenheiro precisa vasculhar repositórios da NASA, ESA e do Departamento de Defesa dos EUA, abrir documentos um a um e julgar se cada um é ou não um documento de concepção. É lento, caro e sujeito a fadiga e erro humano.

Este sistema automatiza a **primeira etapa** desse trabalho: encontrar e baixar os documentos candidatos, de forma reproduzível e com procedência registrada.

1.2 Onde ele se encaixa no pipeline da IC

O projeto de Iniciação Científica tem cinco etapas sequenciais. Este repositório implementa a primeira:

Etapa	O que faz	Situação
1	Coleta automática (<i>Web Crawling</i>)	ESTE SISTEMA — implementado e rodando
2	Extração de texto e pré-processamento	Futura — 87% dos documentos do NTRS já chegam com texto extraído, o que reduz muito o trabalho desta etapa
3	Filtragem por palavras-chave sobre o texto completo	Futura — o <code>lexicon.yaml</code> construído aqui é reaproveitado integralmente
4	Representação semântica (<i>embeddings</i> BERT)	Futura — o pontuador de links foi desenhado para depois usar esses vetores
5	Classificação automática por LLM	Futura — a classe negativa coletada aqui é o que torna a avaliação possível

Por que a Etapa 1 determina a qualidade da Etapa 5

O objetivo científico final é **aferir se um LLM classifica ConOps de forma confiável**. Isso é uma medição, e toda medição depende do conjunto sobre o qual é feita. Se o corpus vier majoritariamente de um único repositório, o classificador aprende a reconhecer *o formato daquele repositório* em vez de reconhecer *o que é um ConOps* — e a métrica final ficaria inflada, sem se sustentar fora do acervo de origem.

Por isso a **diversidade de fonte é requisito de validade experimental**, e não conveniência de engenharia. É esse o argumento que justifica construir um *crawler* genérico em vez de apenas consumir as duas ou três APIs disponíveis.

1.3 O que é um ConOps

ConOps (*Concept of Operations*, ou "Concepção de Operações") é um documento que descreve **como um sistema será operado** — do ponto de vista de quem o usa, não de quem o constrói. Ele responde: quem são os usuários, em que ambiente o sistema opera, quais cenários operacionais existem, o que muda em relação ao sistema atual.

Duas normas definem a estrutura canônica de um ConOps: a **ISO/IEC/IEEE 29148:2011** e a **ANSI/AIAA G-043A-2012**. Elas prescrevem seções como "*current system or situation*", "*justification for and nature of changes*", "*operational scenarios*" e "*summary of impacts*". Esses títulos de seção são um sinal muito mais específico que o termo isolado "concept of operations", e por isso entram no léxico do sistema.

Para a STPA, o ConOps é a fonte primária: é dele que se extraem as perdas a evitar, os perigos e o contexto operacional.

2. Glossário — toda a nomenclatura explicada

Os termos abaixo aparecem no código e no restante deste documento. A maioria vem da literatura de recuperação de informação e engenharia web.

Crawler (rastreador) / Web Crawling

Programa que navega automaticamente por páginas web seguindo os links de uma para outra, como um usuário clicando. "Rastrear um site" é visitar suas páginas de forma sistemática. É o caso geral de coleta, usado quando não existe uma interface melhor.

Harvester (colhedor)

Programa que consome uma *interface estruturada* de um repositório — uma API ou um protocolo — em vez de navegar HTML. É mais rápido, mais confiável e traz metadados melhores. Neste sistema, o NTRS e o ROSA P são colhidos, não rastreados.

API REST

Application Programming Interface no estilo REST: um endereço web que devolve dados estruturados (normalmente JSON) em vez de página HTML. A API do NTRS devolve até 500 registros de documento por requisição, cada um com título, autores, resumo e categoria.

OAI-PMH

Open Archives Initiative Protocol for Metadata Harvesting. Protocolo padronizado que repositórios acadêmicos e governamentais usam para expor seu acervo a colhedores externos. Tem verbos fixos (`ListRecords`, `GetRecord`, `Identify`) e suporta coleta incremental — pedir só o que mudou desde a última vez. O ROSA P do Departamento de Transportes dos EUA expõe um.

Dublin Core

Conjunto padronizado de campos de metadado (`dc:title`, `dc:creator`, `dc:date`, `dc:subject` ...) usado por bibliotecas e repositórios. É o formato que o OAI-PMH do ROSA P devolve.

Adapter (adaptador)

No código, um módulo que sabe conversar com **uma** fonte específica e traduzir o formato dela para o formato interno do sistema. Existe um adaptador para o NTRS, um para o ROSA P e um genérico que embrulha o motor de rastreamento. Todos entregam o mesmo tipo de objeto, então o resto do sistema não precisa saber de onde veio.

DocumentRecord

O **contrato central** do sistema: a estrutura de dados que descreve um documento candidato (título, resumo, autores, URLs, direitos, procedência). Todo adaptador produz `DocumentRecord`; toda camada seguinte consome `DocumentRecord`. É o que permite trocar de fonte sem mexer no resto.

SourceSpec

Arquivo YAML que descreve uma fonte a ser rastreada: por onde começar, até onde ir, o que seguir, o que evitar, qual o orçamento. Adicionar um repositório novo ao sistema custa escrever um desses — nenhum código Python.

Seed (semente)

URL de partida de um rastreamento. É a escolha humana explícita de por onde começar. Ex.: a página de documentação da FAA, ou o PDF do UAM ConOps 2.0.

Frontier (fronteira)

A fila de coisas ainda não processadas. Termo padrão em literatura de *crawling*. Este sistema tem **duas** fronteiras distintas, e confundi-las é o erro conceitual mais comum — a seção 6.1 explica a diferença.

Scope (escopo)

Os limites do rastreamento: quais domínios podem ser visitados, quais caminhos de URL seguir ou evitar, profundidade máxima, orçamento de páginas. Sem escopo, "rastrear faa.gov" vira rastrear a internet inteira, porque um link leva a outro domínio que leva a outro.

Depth (profundidade)

Quantos "cliques" a página está da semente. A semente é profundidade 0; as páginas linkadas nela são 1; as linkadas nessas são 2, e assim por diante.

Anchor (âncora)

O texto clicável de um link. Em `<a href="/doc.pdf">Concept of Operations v2</a>`, a âncora é "Concept of Operations v2". É uma pista valiosa sobre o que há do outro lado do link — quando não é genérica como "clique aqui".

BFS (busca em largura)

Breadth-First Search. Estratégia de navegação que visita todas as páginas de profundidade 1, depois todas as de profundidade 2, e assim por diante — sem julgar relevância. É o comportamento padrão de um *crawler* ingênuo, e neste sistema serve de **linha de base** para comparação.

Focused crawling (rastreamento focado)

Estratégia que **estima a relevância de um link antes de segui-lo** e visita primeiro o que promete mais. Vem de Chakrabarti et al. (1999). É a implementação literal do "Web Crawling semântico" mencionado na introdução do projeto de IC.

Harvest rate (taxa de colheita)

Métrica de eficiência do rastreamento: **documentos relevantes encontrados ÷ páginas HTML baixadas**. Responde "quanto trabalho útil cada requisição produziu?". É o número que compara o rastreamento focado com o BFS.

Sitemap

Arquivo XML que o próprio administrador do site publica listando os endereços de suas páginas. Quando existe, entrega em uma requisição o que o rastreamento levaria centenas para descobrir. Sempre vale tentar antes de navegar.

robots.txt

Arquivo na raiz de um site que declara quais partes os rastreadores automáticos podem ou não visitar. É uma convenção de boa conduta, não um mecanismo de segurança. O sistema lê e respeita esse arquivo em todas as fontes.

User-Agent (UA)

Texto que todo cliente HTTP envia se identificando ao servidor. Um navegador envia algo como `Mozilla/5.0 (Windows NT 10.0...) Chrome/127`. Este sistema envia, por padrão, uma identificação acadêmica com e-mail de contato, para que administradores possam identificá-lo e contatá-lo.

WAF

Web Application Firewall. Camada de proteção (Akamai, Cloudflare) posta na frente de um site, que bloqueia requisições segundo regras genéricas — entre elas, User-Agents que não parecem de navegador. Foi o que impediu o acesso à FAA e à DTIC.

Magic bytes

Os primeiros bytes de um arquivo, que identificam seu formato. Todo PDF começa com `%PDF-`. O sistema valida o tipo pelos *magic bytes* e não pelo cabeçalho `Content-Type`, porque servidores governamentais erram esse cabeçalho com frequência — chegam a anunciar "application/pdf" e devolver uma página de erro HTML.

ETag / If-None-Match

Mecanismo HTTP de cache. O servidor manda um identificador de versão (`ETag`) junto do arquivo; numa reexecução o cliente envia esse identificador de volta e o servidor responde "304 Não Modificado" sem retransmitir o conteúdo. Economiza banda nas coletas repetidas.

SHA-256

Função de *hash* criptográfico: converte qualquer arquivo numa impressão digital de 64 caracteres. Arquivos idênticos produzem o mesmo *hash*; arquivos diferentes, *hashes* diferentes. É como o sistema detecta que o mesmo ConOps veio da FAA e do ROSA P.

Content-addressed storage (armazenamento endereçado por conteúdo)

Guardar o arquivo com o próprio *hash* como nome. Consequências: deduplicação automática, e coleta idempotente — rodar duas vezes não duplica nada.

Idempotência

Propriedade de uma operação que, executada várias vezes, produz o mesmo resultado de executá-la uma vez. Aqui: reexecutar a coleta não duplica documentos nem refaz downloads já concluídos. Verificado em produção — a segunda execução da descoberta no NTRS devolveu "377 vistos, 0 novos".

Rate limiting (limitação de taxa)

Espaçar as requisições no tempo para não sobrecarregar o servidor. O NTRS publica seu limite (500 requisições a cada 15 minutos) em cabeçalhos HTTP, e o sistema lê e obedece a esses cabeçalhos.

Backoff exponencial

Quando uma requisição falha, esperar antes de tentar de novo — e dobrar a espera a cada nova falha (2s, 4s, 8s...). Evita martelar um servidor que já está com problema.

Tier (faixa)

A classificação que o pré-filtro atribui a cada candidato: `strong`, `weak`, `hard_negative` ou `negative_sample`. Determina se o arquivo será baixado e como será usado na validação.

Hard negative (negativo difícil)

Documento que **parece** pertencer à classe alvo mas não pertence. No caso: apresentações do workshop MIT STAMP, densas em terminologia de segurança de sistemas, mas que não são documentos de concepção. São os exemplos mais informativos para validar um classificador — se ele acerta estes, aprendeu o padrão de verdade em vez de detectar jargão.

Playwright

Biblioteca que controla um navegador real sem interface gráfica (*headless*). Necessária para páginas que montam o conteúdo por JavaScript — o HTML entregue pelo servidor vem vazio, e só depois de executar os scripts os links aparecem.

Fixture

Em testes, um arquivo com uma resposta real gravada (um JSON da API do NTRS, um XML do ROSA P). Permite testar o código sem acessar a rede, o que torna os testes rápidos e independentes de os servidores estarem no ar.

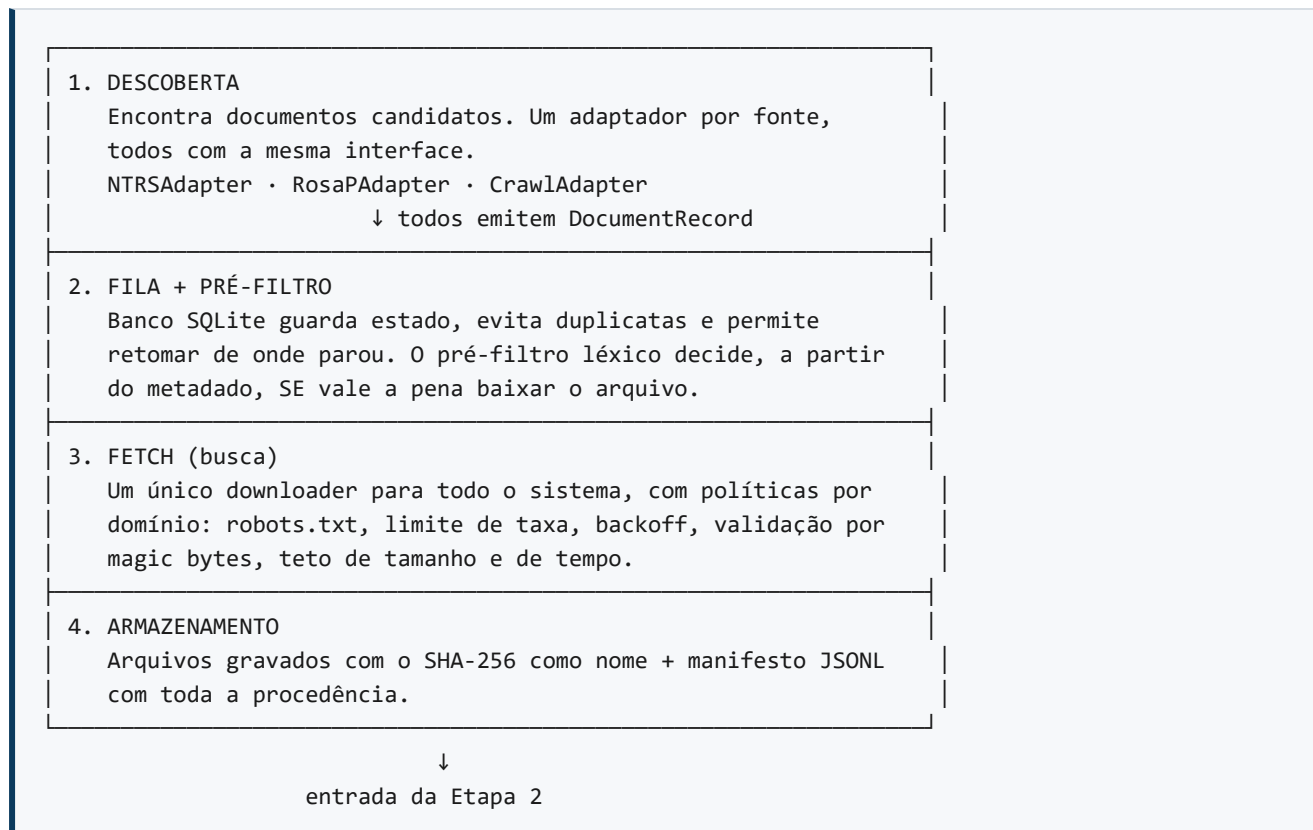
Trap (armadilha de rastreamento)

Padrão de URL que gera páginas infinitas: calendários com link para "próximo mês" para sempre, buscas facetadas onde cada combinação de filtros é um endereço novo, caminhos que se repetem (`/a/b/a/b/a/b...`). Um rastreador que não as detecta nunca termina.

3. Visão geral da arquitetura

3.1 As quatro camadas

O sistema é dividido em quatro camadas desacopladas. "Desacopladas" significa que cada uma conhece apenas o formato de dados que recebe e entrega, não como as outras funcionam por dentro.



Por que separar assim. A camada 1 muda a cada fonte nova; as camadas 2, 3 e 4 nunca mudam. Um adaptador só precisa saber traduzir o formato nativo da sua fonte — limite de taxa, robots.txt, deduplicação e armazenamento não são problema dele.

3.2 As três vias de acesso a uma fonte

Nem toda fonte se acessa do mesmo jeito. O sistema tem três vias, e escolhe a melhor disponível para cada repositório:

Via	Quando usar	Custo por documento
API dedicada	O repositório publica uma API REST documentada. Ex.: NTRS.	Baixíssimo — 1 requisição traz até 500 registros com metadado completo
Protocolo padrão	O repositório expõe OAI-PMH, <i>sitemap</i> ou <i>dumps</i> em CSV. Ex.: ROSA P, FAA, CORDIS.	Baixo — 1 requisição traz 100 registros (OAI-PMH) ou milhares de URLs (<i>sitemap</i>)
Rastreamento HTML	Não há nenhuma das anteriores. É o caso geral . Ex.: MIT PSAS, ESA Cosmos.	Alto — cada requisição traz uma página, da qual se extraem alguns links

A regra do sistema

Interface estruturada quando existe; rastreamento sempre que não. Não é preguiça preferir a API — é uma questão de ordem de grandeza. Colher 4.000 registros do ROSA P por OAI-PMH leva 40 segundos. Rastrear os mesmos 4.000 em HTML levaria horas e impor carga desnecessária a um servidor público.

Mas o rastreamento não é "último recurso": é o que permite ao projeto atacar qualquer repositório novo sem escrever código, e é o que garante a diversidade de fonte de que a Etapa 5 depende.

4. Passo a passo: o que acontece quando você roda um comando

Esta seção acompanha uma execução real do começo ao fim. É a parte mais importante para entender o sistema.

Comando

```
python -m crawler.cli discover faa --max-pages 200
```

"Descubra documentos candidatos na fonte 'faa', gastando no máximo 200 páginas."

Passo 1 — Carregamento da configuração

A CLI (*Command Line Interface*, interface de linha de comando) lê dois arquivos:

- `config/domains.yaml` — as políticas de rede: quanto esperar entre requisições em cada domínio, qual User-Agent usar, tetos de tamanho e tempo, e a *blocklist* de domínios proibidos.
- `config/lexicon.yaml` — o vocabulário que define o que é um ConOps: termos fortes, assinatura estrutural das normas ISO 29148 e AIAA G-043A, marcadores adversariais e pesos.

Passo 2 — Resolução da fonte

A CLI verifica se `faa` é um dos adaptadores dedicados (`ntrs` ou `rosap`). Não é. Então procura `config/sources/faa.yaml` e carrega o `SourceSpec`:

```
seeds:                                # por onde começar
- ../Final_CSINAS_ConOps.pdf
- ../Urban Air Mobility (UAM) Concept of Operations 2.0.pdf
- https://www.faa.gov/uas/advanced_operations
scope:
  allow_hosts: [faa.gov]               # só este domínio
  allow_paths: ['/uas/', '/air_traffic/', '/nextgen/', ...]
  deny_paths: ['/jobs', '/foia', '/search', ...]
  max_depth: 3                         # no máximo 3 cliques da semente
  max_pages: 400                       # orçamento
strategy: focused                      # pontuar links antes de seguir
sitemap: auto                          # tentar sitemap antes de navegar
```

Passo 3 — Semeadura da fronteira de URLs

As sementes entram na fila de URLs com prioridade máxima (1000) — são escolha humana explícita. Cada semente é classificada como **documento** (termina em `.pdf`) ou **página** (a expandir).

Passo 4 — Tentativa de sitemap

Antes de navegar, o sistema procura um *sitemap*: primeiro o declarado no `robots.txt`, depois o endereço padrão `/sitemap.xml`. Na FAA isso devolveu **4.175 URLs** em poucas requisições.

Cada URL do *sitemap* é filtrada pelo escopo, pontuada e enfileirada. Existe um orçamento (`SitemapBudget`) que abandona a tentativa se o rendimento for baixo — no ESA Cosmos, o índice apontava para 87 sub-*sitemaps* de **uma URL cada**, o que consumiria o orçamento inteiro sem proveito.

Passo 5 — O laço principal

Enquanto houver orçamento e URLs pendentes:

1. **Escolher a próxima URL.** A fila devolve a de maior prioridade; em caso de empate, a de menor profundidade. *É aqui que as duas estratégias diferem:* no modo focado as prioridades vêm do pontuador; no BFS todas valem zero, e a ordenação por profundidade produz exatamente busca em largura.
2. **Verificar o orçamento por host** — impede que um único domínio consuma tudo.
3. **Baixar a página** pela camada de *fetch*, que aplica: consulta ao `robots.txt` (com cache), espera do limite de taxa, User-Agent apropriado, *backoff* em caso de erro, e validação do conteúdo por *magic bytes*.
4. **Interpretar o HTML** com BeautifulSoup.
5. **Extrair os metadados do `<head>`** — `citation_*` (padrão Highwire, usado pelo Google Scholar), `DC.*` (Dublin Core), OpenGraph e JSON-LD. É isto que dá a uma fonte HTML metadado comparável ao de uma API. O resultado fica em cache para uso posterior.
6. **Extrair os links**, cada um com sua âncora e o texto ao redor.
7. **Para cada link:** está no escopo? é armadilha? Se sobreviver, o pontuador calcula sua prioridade e ele entra na fila.

Passo 6 — Emissão dos registros

Terminada a travessia, cada documento descoberto vira um `DocumentRecord`. O ponto delicado aqui é **escolher o título**, porque é sobre ele que o pré-filtro decidirá se o documento é relevante:

1. Se a página que continha o link é a *landing page* daquele documento — ou porque declarou `citation_pdf_url` apontando para ele, ou porque continha um único documento — usa-se o metadado da página.
2. Senão, usa-se a âncora, desde que informativa ("UAM Concept of Operations 2.0" serve; "Download PDF" ou uma URL, não).
3. Em último caso, o nome do arquivo, limpo (`uam-conops-2.pdf` → "uam conops 2").

Passo 7 — Pré-filtro e enfileiramento

Cada `DocumentRecord` passa pela *pipeline*:

1. **Controle de exportação:** se o metadado marca ITAR ou EAR, o registro é descartado antes de tudo e a razão é registrada.
2. **Pontuação léxica:** título, resumo e assuntos são comparados ao léxico, resultando numa faixa.
3. **Amostragem de negativos:** documentos sem sinal são amostrados (cerca de 1 em 8, com teto por fonte) — exceto em fontes marcadas como `hard_negatives`, onde tudo é coletado.
4. **Inserção na fila de documentos** (banco SQLite separado), se ainda não conhecido.

Passo 8 — A coleta propriamente dita

Este é um **comando separado**, executado depois:

```
python -m crawler.cli harvest --tier strong weak --limit 100
```

Para cada candidato pendente, o sistema percorre suas URLs **em ordem de preferência**. No NTRS, o `.txt` vem antes do `.pdf` — quando existe, a NASA já entregou o texto extraído e a Etapa 2 não precisa rodar para aquele documento.

Baixado o arquivo: calcula-se o SHA-256, grava-se em `raw/{2 primeiros dígitos}/{sha}.pdf`, valida-se a integridade com pypdf, e registra-se tudo no manifesto. Se o SHA já existia, o arquivo **não** é regravado — apenas acumula-se mais uma procedência.

Por que descoberta e coleta são comandos separados

A descoberta lê **só metadados**: é barata e pode ser reexecutada à vontade quando o léxico mudar. A coleta gasta **banda e disco**. Medir a cobertura do sistema contra um conjunto de referência exige apenas a primeira fase — e é por isso que expandir o vocabulário e testar de novo custa quase nada.

5. Arquitetura detalhada, módulo por módulo

São 3.903 linhas de código de produção, 1.716 de testes e 632 de configuração, distribuídas em três pacotes.

5.1 Núcleo — `crawler/core/`

O que não muda quando se acrescenta uma fonte.

Módulo	Linhas	Responsabilidade
<code>record.py</code>	118	Define <code>DocumentRecord</code> , o contrato entre a Etapa 1 e todo o resto. Também canonicaliza URLs e calcula <i>hashes</i> .
<code>prefilter.py</code>	198	Carrega o léxico, normaliza texto (minúsculas, sem acentos, pontuação virando espaço) e pontua documentos em faixas. Contém também o amostrador determinístico de negativos.
<code>fetcher.py</code>	402	O único downloader do sistema. <code>robots.txt</code> , limite de taxa, <i>backoff</i> , requisições condicionais, validação por <i>magic bytes</i> , tetos de tamanho e de tempo, <i>blocklist</i> .
<code>frontier.py</code>	283	Fila de documentos em SQLite: estado, deduplicação, retomada, e o registro de conteúdos com suas procedências.
<code>store.py</code>	93	Armazenamento endereçado por conteúdo e escrita do manifesto e dos rejeitados.
<code>pipeline.py</code>	264	Orquestra: descoberta → pré-filtro → fila → <i>fetch</i> → armazenamento. Contabiliza as métricas.

Detalhes que valem entender

Validação por *magic bytes*, não por `Content-Type`. Um caso real: a API do NTRS devolve texto extraído com `Content-Type: text/plain`, mas o conteúdo às vezes é XHTML produzido pelo Apache Tika. Outro caso: servidores anunciam "application/pdf" e devolvem uma página de erro HTML com código 200. Só os bytes dizem a verdade.

Teto de tempo além do teto de tamanho. O *timeout* da biblioteca HTTP é por operação de leitura: um servidor que envia bytes em conta-gotas nunca o dispara. Isso travou uma coleta por 20 minutos em 10 documentos, e por isso existe um limite de relógio de parede para o corpo inteiro.

5.2 Motor de rastreamento — `crawler/engine/`

O *crawler* genérico. É a parte maior e mais nova do sistema.

Módulo	Linhas	Responsabilidade
<code>spec.py</code>	170	<code>SourceSpec</code> e <code>Scope</code> — o contrato YAML de uma fonte: sementes, limites, estratégia, papel, orçamento.
<code>urlfrontier.py</code>	223	Fila de URLs em SQLite, ordenada por prioridade e profundidade. Suporta promoção: uma URL reencontrada por um caminho melhor sobe na fila.
<code>linkscorer.py</code>	166	Pontuação semântica de links — o coração do rastreamento focado.
<code>extract.py</code>	284	Extração de links (com âncora e contexto) e de metadados estruturados do <code><head></code> .
<code>sitemap.py</code>	156	Descoberta e leitura de <i>sitemaps</i> , incluindo índices aninhados, com orçamento e desistência.
<code>traps.py</code>	124	Canonicalização de URL e detecção de armadilhas.
<code>crawler.py</code>	451	O laço que junta tudo, mais a contabilidade das métricas.
<code>renderer.py</code>	126	Playwright, ativado apenas nas fontes que declaram precisar.

O que o `extract.py` faz de mais valioso

Repositórios institucionais quase sempre publicam metadados estruturados no cabeçalho HTML. O sistema lê quatro padrões, em ordem de qualidade:

1. **Highwire Press** (`citation_title`, `citation_author`, `citation_pdf_url` ...) — o padrão que o Google Scholar consome. O mais completo.
2. **Dublin Core** (`DC.title`, `DC.creator` ...).
3. **OpenGraph** (`og:title`, `og:description`).
4. **JSON-LD schema.org**.

É isto que equipara uma fonte HTML a uma API. Sem ler essas tags, o *crawler* traria um PDF sem título, sem autor e sem data — inútil para o corpus.

5.3 Adaptadores — `crawler/adapters/`

Módulo	Linhas	Responsabilidade
<code>base.py</code>	37	A interface comum: todo adaptador tem um método <code>discover()</code> que produz <code>DocumentRecord</code> .
<code>ntrs.py</code>	242	Cliente da API da NASA, com particionamento por ano e o job semanal de sincronização exigido pelos termos de uso.
<code>rosap.py</code>	197	Colhedor OAI-PMH do Departamento de Transportes dos EUA.
<code>crawl.py</code>	64	Embrulha o motor de rastreamento na mesma interface dos demais.

Três armadilhas reais da API do NTRS, verificadas contra o serviço

- **O parâmetro de paginação é `page.from`, não `from`.** O parâmetro errado é ignorado *em silêncio* e o servidor devolve sempre a primeira página — produziria um laço infinito colhendo os mesmos registros, sem erro em log nenhum.
- **Parâmetros desconhecidos não geram erro.** Um código HTTP 200 não prova que o filtro foi aplicado. Cada filtro usado foi validado observando mudança na contagem total de resultados.
- **Existe um teto rígido de 10.000 resultados por consulta**, sobre a *soma* de posição e tamanho de página. Sem particionar as consultas por faixa de ano, a cauda de qualquer busca grande se perde em silêncio — perda de cobertura que não aparece em nenhum log.

5.4 Interface de linha de comando — `crawler/cli.py`

305 linhas que expõem cinco comandos: `discover`, `harvest`, `experiment`, `retry-failed`, `report` e `sync-ntrs`. A seção 14 traz a referência completa.

6. Conceitos que costumam confundir

6.1 Por que existem duas filas

Esta é a distinção conceitual mais importante do sistema, e a mais fácil de confundir. São duas coisas diferentes com nomes parecidos:

	Fila de URLs <code>engine/urlfrontier.py</code>	Fila de documentos <code>core/frontier.py</code>
Guarda	Endereços ainda não visitados	Documentos candidatos já identificados
Pergunta que responde	"Qual página visitar agora?"	"Qual arquivo baixar agora?"
Ordenada por	Prioridade do link, depois profundidade	Pontuação léxica do documento
Existe para	Só o rastreamento HTML	Todas as fontes, inclusive as de API
Arquivo	<code>urlfrontier.sqlite</code>	<code>frontier.sqlite</code>

Numa fonte de API como o NTRS, a primeira fila nem chega a existir — não há navegação. Numa fonte rastreada como a FAA, as duas trabalham em sequência: a de URLs conduz a travessia, e o que ela identifica como documento alimenta a de documentos.

6.2 Três identidades diferentes

O sistema distingue três noções de "o mesmo":

1. **Identidade da descoberta** — o par `(fonte, id nativo)`. Evita reprocessar o mesmo registro da mesma fonte. Ex.: `ntrs:20200001712`.
2. **Identidade do conteúdo** — o **SHA-256 do arquivo**. É o que deduplica *entre* repositórios: o mesmo ConOps vindo da FAA, do ROSA P e do NTRS vira um único arquivo em disco, com as três procedências registradas.
3. **Endereço** — a URL. **Nunca é usada como identidade**. No portal Liferay da ESA a URL carrega um identificador único e um *timestamp*, e muda toda vez que o documento é reeditado. Usá-la como chave faria o mesmo documento ser contado várias vezes.

6.3 Descobrir ≠ baixar

A descoberta produz uma *lista de candidatos com metadados*; a coleta produz *arquivos em disco*. A separação existe porque as duas têm custos completamente diferentes: no ROSA P, varrer 4.000 metadados levou 40 segundos e permitiu **não baixar 3.583 PDFs** irrelevantes.

7. O pré-filtro léxico e as quatro faixas

O pré-filtro decide, **a partir do metadado**, se vale baixar o arquivo. É onde se ganha ordem de grandeza em tempo e banda.

O que o léxico contém

Categoria	Conteúdo e função
Termos fortes	Nomes institucionais do artefato: <i>concept of operations</i> , <i>ConOps</i> , <i>operational concept</i> , <i>OpsCon</i> , <i>mission operations concept</i> , <i>concept of employment</i> . No título , levam à faixa strong .
Termos moderados	Sugerem concepção sem nomear o artefato: <i>operational scenario</i> , <i>modes of operation</i> , <i>stakeholder needs</i> . Nunca elevam sozinhos.
Assinatura estrutural	Títulos de seção canônicos das normas ISO/IEC/IEEE 29148:2011 e ANSI/AIAA G-043A-2012 : <i>current system or situation</i> , <i>justification for and nature of changes</i> , <i>summary of impacts</i> . Sinal muito mais específico que o termo isolado — raramente aparecem juntos fora de um documento de concepção real.
Marcadores adversariais	Contêm sinal forte mas não são documentos de concepção: <i>review checklist</i> , <i>lessons learned</i> , <i>meeting minutes</i> , <i>progress report</i> . Rebaixam de strong para weak e marcam para revisão manual.
Vocabulário de safety	<i>STPA</i> , <i>STAMP</i> , <i>hazard analysis</i> , <i>unsafe control action</i> . Não indica ConOps — serve para identificar os negativos difíceis.

As quatro faixas

Faixa	Condição	Ação	Corpus atual
strong	Sinal forte no título, sem marcador adversarial	Baixar sempre	92
weak	Sinal apenas no resumo/assuntos, ou strong rebaixado por marcador adversarial	Baixar e marcar para revisão	224
hard_negative	Fonte declarada como fornecedora de negativos difíceis (MIT PSAS)	Coletar tudo , sem amostrar	627
negative_sample	Sem sinal, em fonte comum	Amostrar ~1 em 8, com teto por fonte	856

Por que coletar a classe negativa não é opcional

O objetivo final é um **classificador binário**. Sem exemplos negativos não há como calcular **precisão** (quantos dos classificados como ConOps realmente são), **recall** (quantos dos ConOps existentes foram encontrados) nem **F1** (a média harmônica dos dois). Sem essas métricas, a Etapa 5 vira demonstração em vez de avaliação.

Há três tipos de negativo, em ordem crescente de valor:

- **Fáceis** — documentos aleatórios sem nenhum sinal.
- **Difíceis** — o acervo MIT PSAS: denso em terminologia de segurança de sistemas, mas não são documentos de concepção.
- **Adversariais** — contêm o termo "ConOps" no título mas são *checklists*, atas ou lições aprendidas. Os mais informativos de todos.

A amostragem é **determinística**: usa uma semente fixa combinada ao identificador do documento, de modo que reexecutar a coleta seleciona exatamente os mesmos negativos. Isso é requisito de reprodutibilidade.

8. O rastreamento focado (*Web Crawling* semântico)

A introdução do projeto de IC menciona "*Web Crawling* semântico". A implementação literal disso na literatura é o **focused crawling** (Chakrabarti et al., 1999): em vez de varrer o site em largura, estima-se a relevância de cada link **antes de segui-lo** e expande-se primeiro o que promete mais.

As três fontes de evidência

Evidência	Peso	Justificativa
Tokens da URL	1,00	O mais confiável: <code>/uam-conops-2.0.pdf</code> diz mais que qualquer âncora, porque quem nomeia um arquivo raramente mente. Exige decodificar o <i>percent-encoding</i> — sem isso, <code>%20</code> esconderia a frase inteira.
Texto da âncora	0,70	Texto humano, mas nem sempre informativo: "Concept of Operations v2" é explícito; "clique aqui" não diz nada.
Contexto ao redor	0,25	Salva quando a âncora é pobre: um "PDF" dentro de um parágrafo sobre concepção operacional vale mais que o mesmo "PDF" numa lista de atas de reunião.

A esses somam-se três ajustes: **bônus** para links que apontam a documentos (são o alvo, não um passo intermediário), **penalidade por profundidade** (impede afundar num único ramo) e **penalidade para ruído estrutural** (`/login`, `/newsroom`, `/careers`).

Como as duas estratégias convivem no mesmo código

A única diferença entre `focused` e `bfs` é o pontuador estar ligado. Com ele desligado, toda prioridade vale zero e a ordenação da fila (*prioridade decrescente, depois profundidade crescente*) degenera exatamente em busca em largura.

Isso é deliberado: **comparar duas estratégias só tem valor se forem idênticas em tudo o mais**.

Resultado medido — FAA, 4 de agosto de 2026

Mesmo orçamento, mesmas sementes, *sitemap* desligado (para que a comparação medisse navegação, e não a lista pronta que o *sitemap* entrega a ambas):

Métrica	BFS	Focado
Páginas baixadas	79	79
Links vistos	15.408	15.507
Documentos encontrados	256	292
Documentos relevantes	2	4
Harvest rate	0,0253	0,0506

Ganho de 2,0× no rendimento por página baixada.

Ressalva honesta, a registrar no relatório

Os números absolutos são **pequenos**: 2 contra 4 documentos relevantes. O resultado é *sugestivo*, não conclusivo — $n=2$ versus $n=4$ não sustenta afirmação estatística. Para o relatório final, é necessário repetir com orçamento de várias centenas de páginas e em mais de uma fonte.

Vale também registrar o contraexemplo observado em teste controlado: quando o ramo relevante é o **primeiro** link da página, o BFS empata com o focado. O ganho aparece quando o conteúdo relevante está *enterrado* — que é o caso real dos portais institucionais.

Evolução prevista para a Etapa 4

Quando a Etapa 4 produzir os *embeddings* BERT, este mesmo componente pode trocar a pontuação léxica por **similaridade vetorial** entre o contexto do link e um protótipo de ConOps, sem alterar o motor — a interface do pontuador foi desenhada para essa substituição. Isso fecha um ciclo entre as etapas e reaproveita o modelo em duas frentes.

9. Inventário de fontes e situação de cada uma

Fonte	Via	Situação	Observações
NTRS NASA STI	API REST	COLETANDO	295 candidatos. 87% dos documentos vêm com texto já extraído pela própria NASA, o que dispensa a Etapa 2 para eles. Fonte âncora do corpus.
ROSA P US DOT	OAI-PMH	COLETANDO	417 candidatos. Acervo em domínio público, coleta incremental nativa. Contém relatórios da FAA, NHTSA e FHWA.
MIT PSAS	Rastreamento	COLETADO	703 documentos, todos <i>hard negatives</i> . Não contém ConOps — contém apresentações <i>sobre</i> STPA. Papel duplo: vocabulário para a Etapa 3 e negativos difíceis para validar a Etapa 5.
FAA	Sitemap + rastreamento	COLETANDO	383 candidatos. O <i>sitemap</i> entregou 4.175 URLs. Requer exceção de User-Agent (ver seção 10).
ESA Cosmos	Rastreamento	POUCO MATERIAL	O rastreamento funciona, mas o material público é escasso: a página do Euclid tem 2 subpáginas e os links de documento são imagens. A documentação de projeto está no Content Server, que exige autenticação e está em <i>blocklist</i> .
ESA EOF	Rastreamento + Playwright	SÓ METADADO	A renderização JavaScript funciona e a categoria "Operations concept" lista exatamente os 2 alvos. Porém as <i>landing pages</i> não contêm link para arquivo algum — os PDFs ficam atrás do visualizador e da área de login. Serve como fonte de metadado, não de arquivo.
DTIC	Sitemap autorizado	INDISPONÍVEL	A DTIC <i>autoriza explicitamente</i> a coleta pelo <i>sitemap</i> , mas o host inteiro está redirecionando para uma página de manutenção. Nenhuma técnica de cliente contorna um servidor desligado — é necessário reverificar periodicamente.
CORDIS	Bulk CSV + rastreamento	NÃO IMPLEMENTADO	O SourceSpec está escrito, mas o adaptador que baixa e filtra os <i>dumps</i> CSV ainda não. É a maior lacuna funcional do sistema hoje.

Estado do corpus

Indicador	Valor
Candidatos descobertos	1.799
Documentos baixados e armazenados	213
Volume em disco	157 MB
URLs rastreadas (todas as fontes)	3.422
Falhas de download pendentes	1
Descartados por controle de exportação	0
Testes automatizados	124

10. Conformidade legal e ética

Seção obrigatória num projeto com financiamento FAPESP e parceria industrial (Embraer).

Fonte	Obrigação verificada	Implementação
NTRS	Citar a fonte; consultar as redistribuições semanalmente e remover documentos retirados; preferir a URL de citação a redistribuir texto integral	Comando <code>sync-ntrs</code> e campo <code>withdrawn_at</code> no manifesto
DTIC	Apenas material com <i>Distribution Statement A</i> ; não tentar o R&E Gateway (exige CAC/PIV)	<code>rndgateway.dtic.mil</code> em <i>blocklist</i>
ESA Cosmos	<code>dms.cosmos.esa.int</code> exige autenticação	<i>Blocklist</i> explícita — não se tenta acesso em hipótese alguma
ROSA P	Domínio público	Sem restrição
MIT PSAS	Material de workshop, direitos dos autores	Uso interno de pesquisa; não redistribuir
Todas	Controle de exportação ITAR/EAR	Descarte antes da fila, com contagem registrada

Postura geral

O corpus é **interno ao projeto**, para treinamento e avaliação. O que se publica são **metadados, métricas e identificadores persistentes** (DOI, id NTRS, número AD) — nunca os PDFs. Isso resolve simultaneamente os termos de uso do NTRS e o risco de direitos autorais.

A exceção de User-Agent

FAA e DTIC estão atrás de um WAF que responde **403 a qualquer User-Agent que não seja exatamente o de um navegador**. Um UA híbrido (navegador com identificação acadêmica anexada) também é bloqueado — não há meio-termo técnico.

Isso derruba a premissa comum de que "administradores toleram bots identificados de universidades": para sites `.gov` atrás de Akamai, o efeito é o inverso.

Decisão do projeto e seus fundamentos

Usar User-Agent de navegador **restrito a** `faa.gov` e `apps.dtic.mil`, a 1 requisição a cada 3–4 segundos. Fundamentos:

1. São **documentos públicos de governo**, publicados com a finalidade de acesso público. Não há conteúdo restrito nem contorno de autenticação.
2. A regra do WAF é **mitigação genérica de bots** aplicada pelo provedor de CDN, não política declarada da instituição.
3. A DTIC **autoriza explicitamente** a coleta do seu acervo, o que torna o bloqueio um efeito colateral da CDN.
4. A carga é desprezível, com orçamento fechado por fonte.

O argumento mais forte, verificado: com o User-Agent de navegador, o `robots.txt` da FAA **passa a ser legível** — e o sistema passa a respeitá-lo, o que antes era tecnicamente impossível. Seu conteúdo é um `robots.txt` padrão de Drupal que proíbe `/core/`, `/profiles/` e `/admin/`, e **permite exatamente os caminhos que o projeto rastreia**.

Ou seja: a política de rastreamento declarada pela FAA autoriza esta coleta, e o WAF era o que impedia tanto a coleta quanto a leitura da política que a permite.

11. O que já está pronto — pontos positivos

Funcionalidade

- **Três vias de acesso funcionando:** API dedicada (NTRS), protocolo padrão (OAI-PMH do ROSA P, *sitemaps*) e rastreamento HTML genérico.
- **Adicionar uma fonte custa um arquivo YAML** — nenhum código Python. Há 8 `SourceSpec` escritos.
- **Renderização JavaScript** disponível e opcional por fonte, com Playwright.
- **Rastreamento focado implementado e medido**, com linha de base BFS para comparação.

Robustez

- **Idempotência verificada em produção:** reexecutar a descoberta devolveu "377 vistos, 0 novos".
- **Retomada após interrupção:** o estado vive em SQLite; uma coleta interrompida continua de onde parou.
- **Deduplicação entre repositórios** por SHA-256, com todas as procedências preservadas.
- **Detecção de armadilhas** de calendário, busca facetada, sessão e caminhos repetidos.
- **Orçamentos em três níveis:** por fonte, por host e por *sitemap*, com desistência automática quando o rendimento é baixo.
- **Validação de conteúdo por *magic bytes***, imune a cabeçalhos `Content-Type` incorretos.

Qualidade e reprodutibilidade

- **124 testes automatizados** que não tocam a rede — usam respostas reais gravadas. Reproduzir a coleta não depende de os servidores estarem no ar.
- **Quatro bugs de campo travados por teste de regressão** (ver detalhes abaixo).
- **Configuração versionada em YAML**, métricas datadas em `reports/`, manifesto JSONL com procedência completa.
- **Amostragem determinística** da classe negativa.

Eficiência medida

- **O pré-filtro evita a maior parte do tráfego:** no ROSA P, 4.000 metadados varridos em 40 segundos permitiram *não* baixar 3.583 PDFs.
- **87% dos documentos do NTRS chegam com texto já extraído**, ocupando 13 MB — os mesmos em PDF ocupariam ordens de grandeza mais e exigiriam extração na Etapa 2.
- **Rastreamento focado dobrou o rendimento** na FAA com orçamento idêntico.

Bugs encontrados apenas contra sites reais

Vale registrar porque ilustram a diferença entre código que compila e código que funciona:

1. `href` **malformatado derrubava o rastreamento inteiro**. Um *placeholder* de template (`[auth_url]`) fazia a biblioteca tentar interpretá-lo como endereço IPv6 e levantar exceção. Um link ruim numa página

custava a coleta inteira da fonte.

2. **Título vazando de página de índice.** Os 40 PDFs de uma página de listagem herdavam todos o título da listagem. Isso corromperia o metadado do corpus e contaminaria a avaliação da Etapa 5.
3. **[http://](#) e [https://](#) contados como documentos distintos**, dobrando a contagem e gastando o orçamento duas vezes no mesmo arquivo.
4. **Relevância julgada cedo demais.** No ROSA P o arquivo se chama [dot_78914_DS1.pdf](#) e não carrega sinal algum — o título está na *landing page*. Julgar antes de resolver o metadado subestimava tudo.

12. Limitações e pontos negativos

Registro honesto do que ainda não funciona ou funciona mal.

12.1 Lacunas funcionais

Lacuna	Gravidade	Detalhe
Adaptador CORDIS não implementado	ALTA	O <code>SourceSpec</code> existe, o código não. É a única fonte planejada sem implementação, e representa a diversidade europeia do corpus (projetos SESAR JU, ConOps de gerenciamento de tráfego aéreo).
Não existe <i>gold set</i>	ALTA	Sem um conjunto de referência rotulado manualmente, não é possível medir a cobertura (<code>recall@gold</code>) — a métrica-chave da Etapa 1. Hoje só se sabe quantos documentos foram coletados, não que fração dos existentes foi encontrada.
Precisão do pré-filtro não medida	MÉDIA	Não se sabe que fração dos <code>strong</code> é realmente ConOps, nem quanto dos <code>weak</code> se aproveita. Exigiria revisão manual de ~100 itens de cada faixa.
Experimento com amostra pequena	MÉDIA	O ganho de 2,0× do rastreamento focado baseia-se em 4 contra 2 documentos relevantes. Sugestivo, não conclusivo.
Etapa 2 ausente	MÉDIA	Fora do escopo deste repositório, mas é a dependência imediata: 13% dos documentos do NTRS e todos os PDFs das demais fontes precisam de extração de texto.

12.2 Limitações técnicas

- **Sem paralelismo.** A configuração prevê um campo de concorrência por domínio, mas o downloader é sequencial. Coletar milhares de documentos leva horas que poderiam ser minutos. O SQLite com escritor único também limita quanto se pode paralelizar sem mudar a arquitetura de estado.
- **Corpo inteiro carregado em memória.** O teto de 60 MB por arquivo protege, mas um documento grande ocupa toda a sua memória durante o download em vez de ser gravado em fluxo.
- **Deduplicação apenas exata.** Dois PDFs do mesmo documento com um byte de diferença (metadado de geração, marca d'água, versão reexportada) contam como documentos distintos. Falta deduplicação aproximada por similaridade.
- **Sem detecção de PDF escaneado.** Um PDF que contém só imagens passará adiante e produzirá texto vazio na Etapa 2, sem aviso.
- **Pontuador de links é léxico, não semântico.** "Web Crawling semântico" está implementado na estrutura (o pontuador existe e funciona), mas a pontuação ainda é casamento de termos, não similaridade vetorial. A ponte para a Etapa 4 está desenhada, não construída.
- **Cobertura de testes não medida formalmente.** São 124 testes e a proporção de código de teste para produção é boa (1.716 para 3.903 linhas), mas não há relatório de cobertura.
- **O job semanal do NTRS não está agendado.** O comando existe, mas depende de execução manual — e os termos de uso o exigem periodicamente.

12.3 Limitações das fontes (fora do controle do projeto)

- **DTIC indisponível** por manutenção do servidor. Sem previsão.
- **ESA EOF não expõe os arquivos** publicamente — só as fichas de metadado.
- **ESA Cosmos tem pouco material público**; a documentação real exige autenticação.
- **Busca do ROSA P bloqueada** para clientes automatizados, embora o OAI-PMH funcione (e seja a via correta de qualquer forma).

13. Próximos passos priorizados

Ordenados por relação entre impacto no projeto de IC e esforço.

Prioridade 1 — Desbloqueiam a avaliação

1. **Construir o *gold set*.** Curar manualmente 50–80 documentos rotulados: ~40 ConOps confirmados, ~20 quase-ConOps (requisitos, descrições de sistema — a fronteira difícil) e ~20 negativos claros.
Por que primeiro: é o que permite medir `recall@gold`, a métrica-chave da Etapa 1. E serve de referência para as Etapas 1, 3 e 5 — investimento que se paga três vezes. Exige julgamento humano; não pode ser automatizado.
2. **Medir a precisão do pré-filtro.** Revisar manualmente 100 itens da faixa `strong` e 100 da `weak`.
Por quê: transforma "coletei 316 documentos" em "coletei 316 documentos com X% de precisão na faixa forte" — a diferença entre relatório descritivo e resultado.
3. **Repetir o experimento focado vs BFS com orçamento grande** (várias centenas de páginas) e em mais de uma fonte.
Por quê: o resultado atual é sugestivo mas frágil. Com amostra maior vira resultado publicável.

Prioridade 2 — Completam a cobertura de fontes

4. **Implementar o adaptador *bulk* do CORDIS.** Baixar os *dumps* CSV mensais, filtrar *offline* por programa (SESAR JU e *clusters* aeroespaciais) e só então baixar os PDFs aprovados.
Por quê: é a única fonte planejada sem código, e traz diversidade europeia — importante para a validade da Etapa 5.
5. **Reverificar a DTIC periodicamente** e coletar assim que sair da manutenção.
6. **Adicionar EASA e NHTSA** como `SourceSpec`, agora que o motor genérico existe. Custo: um YAML cada.

Prioridade 3 — Eficiência

7. **Paralelismo por domínio.** Baixar de várias fontes simultaneamente, respeitando o limite de taxa de cada uma independentemente.
Ganho estimado: de horas para minutos numa coleta completa. Requer revisar o acesso concorrente ao SQLite.
8. **Gravação em fluxo** em vez de carregar o corpo inteiro em memória.
9. **Deduplicação aproximada** por SimHash ou MinHash sobre o texto, para pegar reexportações do mesmo documento.
10. **Deteção de PDF escaneado** na ingestão — contar caracteres extraídos por página e sinalizar os que precisarão de OCR.

Prioridade 4 — Ponte com as etapas seguintes

11. **Pontuador de links por *embeddings*.** Assim que a Etapa 4 produzir o modelo BERT, substituir a pontuação léxica por similaridade vetorial entre o contexto do link e um protótipo de ConOps.

Por quê: é o que torna o "Web Crawling semântico" semântico de fato, fecha um ciclo entre as etapas e reaproveita o modelo. Bom material de artigo.

12. **Agendar o *job* semanal do NTRS** — exigência dos termos de uso, hoje dependente de execução manual.
13. **Relatório de cobertura de testes** e integração contínua, se o projeto for para repositório compartilhado.

Sugestão de sequência para as próximas semanas

Os três itens de prioridade 1 são independentes entre si e podem ser feitos em paralelo com o item 4 (CORDIS). O *gold set* é o único que exige exclusivamente trabalho humano, então convém começá-lo imediatamente — enquanto ele é curado, o desenvolvimento do CORDIS avança sem bloqueio.

14. Referência rápida de comandos

Instalação

```
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements-dev.txt
.venv/Scripts/python.exe -m playwright install chromium # só se for usar render JS
```

Descoberta (barata — só metadados)

```
# Fontes com adaptador dedicado
python -m crawler.cli discover ntrs --year-start 2015 --year-end 2026
python -m crawler.cli discover rosap --max-pages 40 --from 2026-01-01

# Fontes com SourceSpec (rastreamento)
python -m crawler.cli discover faa --max-pages 200
python -m crawler.cli discover psas --max-pages 120
python -m crawler.cli discover esa_cosmos --max-pages 300

# Opções úteis
--reset          zerar a fronteira de URLs antes (recomeçar do zero)
--strategy bfs   forçar busca em largura
--no-sitemap     forçar navegação em vez de usar o sitemap
--no-render      desligar Playwright nesta execução
--limit N        parar após N candidatos novos
```

Coleta (gasta banda e disco)

```
python -m crawler.cli harvest --tier strong weak --limit 100
python -m crawler.cli harvest --tier strong # só os de sinal forte
python -m crawler.cli retry-failed # reenfileirar os que falharam
```

Métricas e experimentos

```
python -m crawler.cli report # métricas do corpus
python -m crawler.cli experiment faa --max-pages 200 # focado vs BFS
python -m crawler.cli sync-ntrs --since 2026-07-28 # exigido pelos termos do NTRS
```

Testes

```
.venv/Scripts/python.exe -m pytest tests/ -q
```

Onde ficam as coisas

Caminho	Conteúdo
<code>config/domains.yaml</code>	Políticas de rede por domínio, <i>blocklist</i> , raiz dos dados
<code>config/lexicon.yaml</code>	Vocabulário de ConOps — reaproveitado nas Etapas 3 e 5
<code>config/sources/*.yaml</code>	Um <code>SourceSpec</code> por fonte rastreada
<code>crawler/core/</code>	Núcleo: registro, pré-filtro, <i>fetch</i> , fila, armazenamento, orquestração
<code>crawler/engine/</code>	Motor de rastreamento
<code>crawler/adapters/</code>	Um adaptador por tipo de fonte
<code>tests/</code>	124 testes e <i>fixtures</i> gravadas
<code>docs/</code>	Achados de campo e documentação do motor
<code>reports/</code>	Métricas datadas de cada execução
<code>C:\ConOpsCorpus\</code>	O corpus — fora do OneDrive, para não sincronizar dezenas de GB

Documentos complementares

- `README.md` — visão geral e resultados
- `docs/crawler.md` — projeto detalhado do motor de rastreamento
- `docs/achados-api.md` — todas as divergências verificadas contra as APIs e sites reais, com evidência